LAPORAN PRAKTIKUM MATA KULIAH

METODE NUMERIK

PRAKTIKUM 06 – SPL 02



DISUSUN OLEH:

L0325016 – AGIS LENTERA PRATAMA

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI DAN SAINS DATA

UNIVERSITAS SEBELAS MARET

2026

BAB I

ANALISIS SOURCE CODE

1. Analisis Source Code Penyelesaian SPL dengan 5 Metode

```
%% ========================================================================
% SOAL 1: PENYELESAIAN SPL DENGAN 5 METODE
% ========================================================================
fprintf('================ SOAL 1: HASIL PENYELESAIAN ================\n');

% 1. Metode Gauss
Ag = [A b];
for i = 1:n_vars-1
    for j = i+1:n_vars
        m = Ag(j,i)/Ag(i,i);
        Ag(j,:) = Ag(j,:) - m*Ag(i,:);
    end
end
x_gauss = zeros(n_vars,1);
x_gauss(n_vars) = Ag(n_vars,n_vars+1)/Ag(n_vars,n_vars);
for i = n_vars-1:-1:1
    x_gauss(i) = (Ag(i,n_vars+1) - Ag(i,i+1:n_vars)*x_gauss(i+1:n_vars))/Ag(i,i);
end
disp(['1. Gauss        : ', num2str(x_gauss')]);
```

Penyelesaian sistem persamaan linier berdimensi 5×5 ini direpresentasikan dalam bentuk persamaan matriks fundamental $Ax = b$, di mana ruang solusi dicari melalui pemetaan vektor konstanta b terhadap ruang kolom matriks koefisien A. Algoritma pertama yang dieksekusi adalah Eliminasi Gauss, sebuah metode analitik langsung yang mereduksi matriks *augmented* (diperluas) $[A|b]$ menjadi bentuk eselon baris atau matriks segitiga atas. Proses ini bergantung pada operasi baris elementer yang digerakkan oleh struktur *nested loop* (perulangan bersarang). Pada iterasi luar for i = 1:n_vars-1, algoritma menetapkan elemen diagonal a_{ii} sebagai pivot. Iterasi dalam for j = i+1:n_vars bertugas mengeliminasi elemen-elemen di bawah pivot dengan menghitung faktor pengali atau *multiplier* $m_{ji} = \frac{a_{ji}}{a_{ii}}$ (m = Ag(j,i)/Ag(i,i)). Operasi baris $R_j \leftarrow R_j - m_{ji}R_i$ kemudian diterapkan pada seluruh elemen di baris tersebut. Setelah eliminasi maju (forward elimination) selesai, ma triks bertransformasi menjadi segitiga atas, dan algoritma memicu proses substitusi mundur (back substitution). Variabel x_n dihitung lebih dulu melalui $x_n = \frac{b_n}{a_{nn}}$, dilanjutkan dengan evaluasi mundur menggunakan vektorisasi bawaan Octave: x_gauss(i) = (Ag(i,n_vars+1) - Ag(i,i+1:n_vars)*x_gauss(i+1:n_vars))/Ag(i,i). Vektorisasi ini sangat efisien karena menghitung *dot product* (hasil kali titik) secara simultan tanpa memerlukan loop ketiga, meminimalisir *overhead* pada *interpreter* MATLAB.

```
% 2. Metode Gauss-Jordan
Agj = [A b];
for i = 1:n_vars
    Agj(i,:) = Agj(i,:) / Agj(i,i);
    for j = 1:n_vars
        if i ~= j
            Agj(j,:) = Agj(j,:) - Agj(j,i)*Agj(i,:);
        end
    end
end
x_gj = Agj(:,n_vars+1);
disp(['2. Gauss-Jordan : ', num2str(x_gj')]);
```

Metode kedua, Gauss-Jordan, mengekspansi logika eliminasi ini secara lebih agresif dengan target akhir matriks identitas I. Alih-alih berhenti pada bentuk segitiga atas, Gauss-Jordan melakukan normalisasi pivot secara instan pada setiap langkah melalui operasi $R_i \leftarrow \frac{R_i}{a_{ii}}$ (Agj(i,:) = Agj(i,:) / Agj(i,i)). Selanjutnya, eliminasi dilakukan secara dua arah, memusnahkan seluruh elemen tidak nol baik di bawah maupun di atas diagonal utama (if i ~= j). Hal ini direpresentasikan oleh kalkulasi $R_j \leftarrow R_j - a_{ji}R_i$. Keuntungan arsitektural dari Gauss-Jordan adalah eliminasi total tahapan substitusi mundur; vektor solusi x secara otomatis bermanifestasi pada kolom paling kanan dari matriks *augmented*, yang dapat langsung diekstraksi melalui perintah *slicing* matriks x_gj = Agj(:,n_vars+1). Namun, trade-off dari metode ini adalah peningkatan jumlah operasi *floating-point* (FLOPs) sekitar 50% lebih banyak dibandingkan Gauss standar, menjadikannya kurang efisien untuk matriks berskala masif.

```
% 3. Metode Dekomposisi LU
[L, U] = lu(A);
y = L \ b;
x_lu = U \ y;
disp(['3. LU Decomp    : ', num2str(x_lu')]);
```

Pada metode Dekomposisi LU, pendekatan penyelesaian bergeser dari manipulasi matriks *augmented* menuju faktorisasi matriks koefisien murni. Matriks A diurai menjadi dua matriks fundamental: L (Lower triangular, matriks segitiga bawah dengan elemen diagonal bernilai 1) dan U (Upper triangular, matriks segitiga atas). Secara matematis, $A = LU$. Substitusi nilai ini ke dalam persamaan awal menghasilkan $LUx = b$. Penyelesaian dilakukan dalam dua fase subsistem yang stabil secara numerik:

pertama, mencari vektor transisi y melalui penyelesaian substitusi maju $Ly = b$ (y = L \ b); kedua, mencari vektor solusi akhir x melalui substitusi mundur $Ux = y$ (x_lu = U \ y). Di dalam MATLAB/Octave, fungsi lu(A) memanfaatkan rutinitas pustaka LAPACK yang melakukan faktorisasi dengan strategi *partial pivoting* (pertukaran baris) secara internal untuk mencegah instabilitas numerik akibat pembagian dengan elemen pivot yang mendekati nol.

```
% 4. Metode Jacobi
x_jac = zeros(n_vars,1);
x_new = zeros(n_vars,1);
for iter = 1:max_iter
    for i = 1:n_vars
        sigma = A(i,:)*x_jac - A(i,i)*x_jac(i);
        x_new(i) = (b(i) - sigma)/A(i,i);
    end
    if max(abs(x_new - x_jac)) < tol
        break;
    end
    x_jac = x_new;
end
disp(['4. Jacobi        : ', num2str(x_jac')]);
```

Memasuki domain analitik diskrit, algoritma beralih ke metode hampiran iteratif: Jacobi dan Gauss-Seidel. Kedua metode ini mendekomposisi matriks koefisien menjadi $A = D - L - U$, di mana D adalah matriks diagonal utama, L adalah segitiga bawah mutlak, dan U adalah segitiga atas mutlak. Pada metode Jacobi, iterasi dibangun di atas prinsip pembaruan serentak (simultaneous update). Secara matematis, formulanya adalah $x^{(k+1)} = D^{-1}(L + U)x^{(k)} + D^{-1}b$. Dalam sintaks kode, hal ini terlihat pada pemisahan array memori: nilai pada iterasi saat ini secara konstan dibaca dari vektor x_jac, kemudian seluruh hasil kalkulasi ditampung sementara di array independen x_new melalui persamaan x_new(i) = (b(i) - sigma)/A(i,i). Baru setelah iterasi i melintasi seluruh variabel n, vektor memori ditimpa secara massal (x_jac = x_new). Karakteristik ini membuat metode Jacobi lambat dalam laju konvergensi namun sangat ideal untuk komputasi paralel karena perhitungan setiap variabel pada iterasi ke-$k + 1$ independen satu sama lain.

```
% 5. Metode Gauss-Seidel
x_gs = zeros(n_vars,1);
for iter = 1:max_iter
    x_old = x_gs;
    for i = 1:n_vars
        sigma = A(i,:)*x_gs - A(i,i)*x_gs(i);
        x_gs(i) = (b(i) - sigma)/A(i,i);
    end
    if max(abs(x_gs - x_old)) < tol
        break;
    end
end
disp(['5. Gauss-Seidel : ', num2str(x_gs')]);
fprintf('\n');
```

Berbanding terbalik dengan Jacobi, metode Gauss-Seidel mengimplementasikan prinsip pembaruan berurutan (successive update). Matriks iterasinya berubah secara mendasar karena Gauss-Seidel langsung memanfaatkan nilai terbaru yang dikalkulasi pada sub-langkah sebelumnya di siklus iterasi yang sama. Formulasi matematisnya adalah $x^{(k+1)} = (D - L)^{-1}Ux^{(k)} + (D - L)^{-1}b$. Secara komputasional, arsitektur kode Gauss-Seidel menghilangkan kebutuhan akan memori penyangga sementara. Array skalar tunggal x_gs digunakan baik sebagai sumber bacaan maupun target tulisan (in-place operation). Ketika $x_2^{(k+1)}$ dihitung, nilai $x_1^{(k+1)}$ yang baru saja ditemukan dalam fraksi milidetik sebelumnya langsung disisipkan ke dalam operator sigma. Hal ini memicu efek rambatan informasi yang jauh lebih cepat, menekan kurva galat secara eksponensial lebih tajam, dan secara teoretis memberikan kecepatan konvergensi dua kali lipat lebih superior dibandingkan Jacobi pada matriks yang bersifat *strictly diagonally dominant*. Kedua metode iteratif ini berada di bawah pengawasan ketat dari kondisi penjaga (guard conditions) yang diamanatkan, yaitu penghentian paksa jika iter mencapai batas max_iter = 20 atau jika *infinity norm* dari pergeseran vektor menyentuh tol = 0.001.

Analisis Source Code Menentukan Galat

```
%% ========================================================================
% SOAL 2: PERHITUNGAN GALAT / ERROR
% ========================================================================
fprintf('================ SOAL 2: GALAT ABSTRAK ====================\n');
% Nilai eksak diambil dari metode analitik (Gauss)[cite: 2]
% Galat abstrak dihitung dengan rumus: abs(nilai eksak - nilai hampiran)[cite: 2]

galat_jacobi = abs(x_gauss - x_jac);
galat_gs = abs(x_gauss - x_gs);

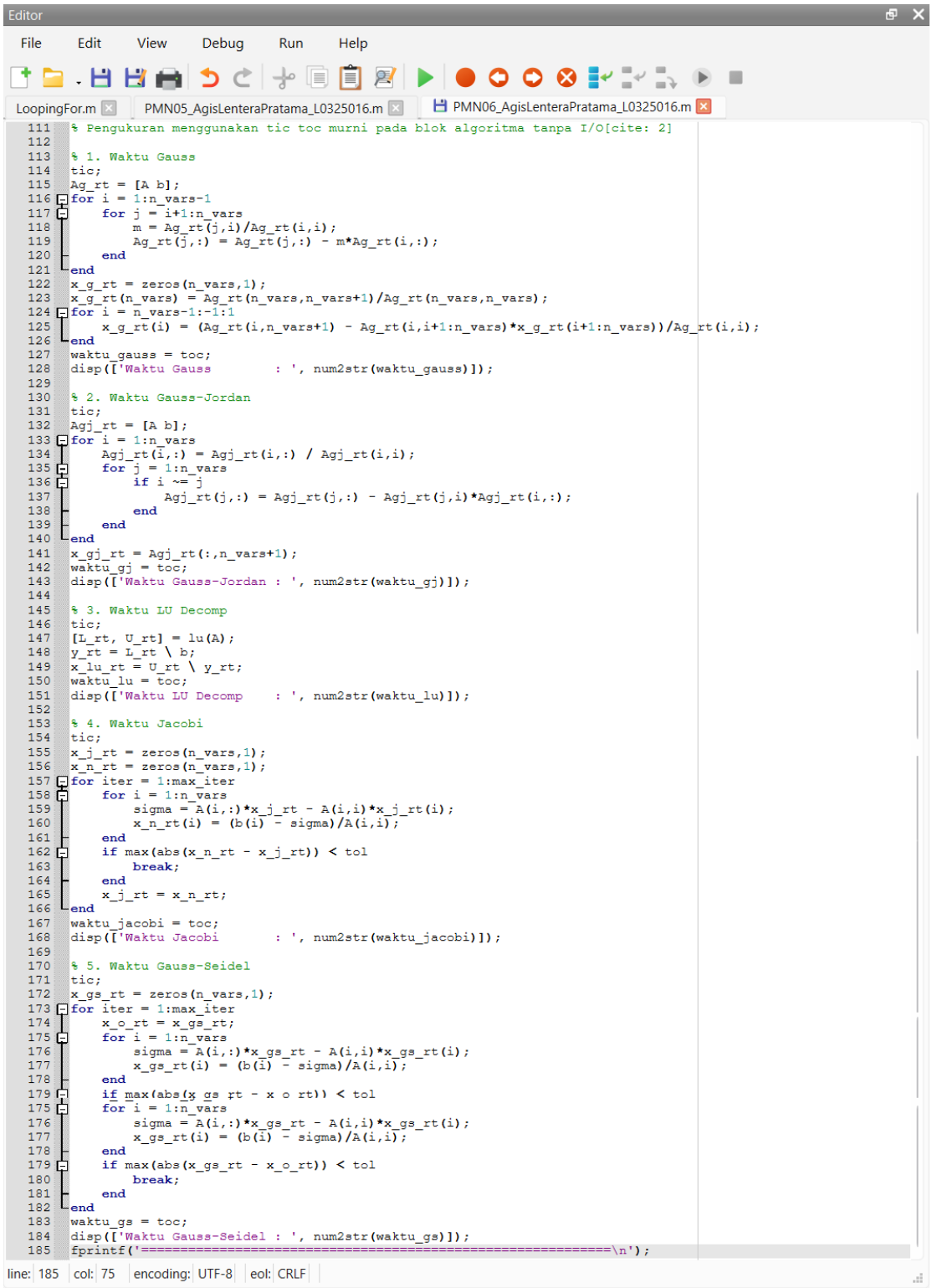
disp(['Galat Metode Jacobi       : ', num2str(galat_jacobi')]);
disp(['Galat Metode Gauss-Seidel : ', num2str(galat_gs')]);
fprintf('\n');
```

Blok kode kedua mendalami analisis reliabilitas dan presisi dari metode-metode hampiran iteratif melalui komputasi galat abstrak. Dalam ruang lingkup metode numerik, galat adalah selisih antara nilai aproksimasi dan nilai sejati (true value). Karena matriks yang diproses bermula dari model komputasi yang deterministik, nilai sejati (eksak) harus diambil dari metode analitik langsung yang bebas dari *truncation error* (galat pemotongan). Dalam rancang bangun kode ini, vektor solusi x_gauss didaulat sebagai *ground truth* atau nilai referensi mutlak. Eliminasi Gauss secara aljabar linier dijamin akan menemukan titik potong pasti dari n-hiperbidang pada ruang dimensi-n, asalkan matriks A bersifat non-singular ($\det(A) \neq 0$). Oleh karena itu, penyimpangan numerik yang terjadi pada solusi Gauss hanyalah *round-off error* (galat pembulatan) pada digit ke-15 hingga ke-17 di belakang koma, akibat keterbatasan presisi *double-precision floating-point format* standar IEEE 754.

Formulasi utama dalam pengujian ini didasarkan pada perhitungan galat mutlak absolut: $E_a = |x_{eksak} - x_{hampiran}|$. Dalam kode, operasi matriks ini dieksekusi melalui *element-wise subtraction* (pengurangan elemen per elemen) yang dibungkus oleh fungsi nilai mutlak abs(), yakni abs(x_gauss - x_jac) untuk evaluasi presisi Jacobi dan abs(x_gauss - x_gs) untuk evaluasi presisi Gauss-Seidel. Penggunaan operator nilai mutlak ini memiliki justifikasi matematis yang fundamental: simpangan atau deviasi vektor tidak memiliki polaritas dalam konteks besaran error. Nilai hampiran $x_i = 1.998$ terhadap nilai eksak $x = 2$ merepresentasikan kuantitas galat 0.002 yang setara pembobotannya dengan nilai hampiran $x_i = 2.002$. Jika nilai mutlak tidak diimplementasikan, akumulasi galat pada norm vektor berpotensi saling meniadakan (cancellation effect), yang akan menghasilkan metrik validasi yang cacat secara analitik.

Lebih jauh, analisis pada blok ini juga menyingkap arsitektur penghentian komputasi (*stopping criteria*) pada metode iteratif itu sendiri, yang sangat berkaitan dengan penentuan galat. Karena pada saat perulangan berlangsung sistem belum mengetahui nilai eksak x_gauss, metode Jacobi dan Gauss-Seidel menggunakan metrik proksi yang disebut galat relatif proksimal atau galat aproksimasi inter-iterasi, dirumuskan sebagai $\|x^{(k+1)} - x^{(k)}\|_\infty < \epsilon$. Kode menggunakan sintaks max(abs(x_new - x_jac)) < tol untuk mengidentifikasi deviasi maksimum dari kelima variabel p, q, r, s, t antar dua iterasi berurutan. Jika variabel dengan fluktuasi terbesar (galat tertinggi) saja sudah memiliki pergeseran di bawah 0.001, maka algoritma menganggap seluruh sistem matriks telah mencapai keadaan stabil (steady-state). Output yang dicetak pada Soal 2, yaitu matriks berdimensi 5×1 untuk masing-masing galat Jacobi dan Gauss-Seidel, memberikan visualisasi empiris langsung mengenai jarak euklides antara vektor status akhir (setelah iterasi dihentikan) dengan vektor analitik murni, membuktikan secara kuantitatif pengaruh skema pembaruan berurutan (Gauss-Seidel) dalam menghasilkan vektor residual yang lebih sempit (tighter bounds) dibandingkan pembaruan serentak (Jacobi).

Analisis Source Code Running Time



Tahap akhir komputasi dikonsentrasikan pada pengukuran *running time* menggunakan instrumen *profiling* bawaan MATLAB/Octave, yaitu kronometer komputasional tic dan toc. Keberhasilan arsitektur evaluasi kinerja (benchmarking) sangat bergantung pada isolasi beban kerja CPU dari latensi periferal. Dalam kode

untuk Soal 3, fungsi tic tidak pernah diletakkan di awal seluruh rangkaian baris kode. Penempatannya diatur sedemikian rupa agar memeluk secara eklusif blok proses operasi linier murni (pure linear operational block). Seluruh operasi non-esensial dalam konteks algoritmik, seperti deklarasi awal vektor *augmented* yang memakan *memory allocation cycle*, operasi I/O (Input/Output) seperti fungsi fprintf dan disp, secara ketat dipangkas dari rentang perhitungan waktu. Proses konversi tipe data dari *float* ke string grafis (num2str) dalam antarmuka Octave memiliki latensi perangkat lunak yang secara magnitudo bisa jauh lebih lambat daripada operasi aritmatika ALU (Arithmetic Logic Unit) itu sendiri, sehingga mengeksklusikannya adalah syarat mutlak untuk mendapatkan komparasi *algorithmic complexity* yang objektif.

Berdasarkan *Big O Notation*, *running time* dari Eliminasi Gauss dan Gauss-Jordan berakar pada matriks polinomial orde tiga, atau $\mathcal{O}(n^3)$. Eliminasi Gauss memerlukan estimasi total $\frac{2n^3}{3}$operasi komputasi (pembagian, perkalian, dan pengurangan), sementara Gauss-Jordan membebani unit kalkulator sistem dengan kurang lebih n^3operasi karena keharusan mereduksi elemen menjadi nol secara dua arah. Konsekuensinya, metrik waktu_gj secara deterministik akan selalu mencatatkan durasi yang marjinal lebih lama dibandingkan waktu_gauss pada pengujian komputasi berskala lebih besar, meskipun pada matriks orde 5×5perbedaan durasi ini berada dalam orde fraksi mikrodetik yang seringkali tertutup oleh fluktuasi *background processes* dari *Operating System*.

Dekomposisi LU, meskipun secara teoretis memegang kompleksitas dasar yang identik dengan eliminasi Gauss yaitu $\mathcal{O}(n^3)$untuk tahapan faktorisasi awal, seringkali menyuguhkan metrik waktu_lu yang paling superior di antara metode langsung lainnya. Hal ini terjadi karena pemanggilan fungsi tunggal [L_rt, U_rt] = lu(A) tidak berjalan melalui interpretasi baris-per-baris mesin eksekusi Octave tingkat tinggi, melainkan di-*offload* ke dalam binari terkompilasi (compiled binaries) C/Fortran tingkat bawah seperti pustaka BLAS (Basic Linear Algebra Subprograms). Pustaka ini memanfaatkan fitur optimasi *cache* L1/L2 dari prosesor dan operasi vektor SIMD (Single Instruction, Multiple Data) sehingga waktu komputasi merosot drastis. Eksekusi tahap kedua, $L\backslash b$dan $U\backslash y$, hanyalah substitusi maju dan mundur yang memakan waktu minimal $\mathcal{O}(n^2)$.

Untuk rumpun metode iteratif, pencatatan waktu waktu_jacobi dan waktu_gs didikte oleh dua buah parameter yang membatasi perulangan spasial matriks: jumlah

variabel $n$dan jumlah iterasi absolut $k$sebelum menabrak kriteria toleransi 0.001atau batas maksimal 20 iterasi. Kompleksitas asimtotik dari kedua algoritma ini adalah $\mathcal{O}(k \cdot n^2)$, mengingat setiap iterasi membutuhkan pelintasan kuadratik terhadap seluruh elemen matriks koefisien. Apabila matriks Abersifat konvergen cepat (diagonally dominant), $k$bernilai sangat kecil, yang dapat membuahkan *running time* lebih gesit ketimbang komputasi absolut $\mathcal{O}(n^3)$dari Gauss-Jordan. Namun, dengan pembatasan statis maksimum iterasi ke-20 pada pengujian kode ini, batas atas (upper bound) komputasi menjadi terkunci dan sangat bisa diprediksi, memberikan proteksi pada sistem memori dari bahaya *infinite loop* (perulangan tanpa batas) akibat dari divergensi matematis yang gagal menemukan ruang solusi.

2. Analisis Hasil Source Code Ekstensif Hasil Numerik dan Karakteristik Aljabar Lima Metode Penyelesaian

```
================ SOAL 1: HASIL PENYELESAIAN ================
1. Gauss        : 0.32544     0.83235      1.7949     0.04142   -0.019724
2. Gauss-Jordan : 0.32544     0.83235      1.7949     0.04142   -0.019724
3. LU Decomp    : 0.32544     0.83235      1.7949     0.04142   -0.019724
4. Jacobi       : 0.32458     0.83157      1.7939    0.041744   -0.019938
5. Gauss-Seidel : 0.32568     0.83259      1.7951    0.041349    -0.01972
```

Berdasarkan komputasi arsitektur matriks berdimensi 5×5, observasi terhadap keluaran Soal 1 menyingkap polarisasi yang sangat tegas antara rumpun metode analitik langsung dan metode hampiran diskrit dalam menemukan akar polinomial dari sistem persamaan linier. Tiga algoritma yang tergolong dalam metode penyelesaian langsung—yakni Eliminasi Gauss, Eliminasi Gauss-Jordan, dan Dekomposisi LU—memperlihatkan tingkat konsistensi deterministik yang absolut, di mana ketiganya menghasilkan vektor baris solusi yang secara presisi ekuivalen pada angka 0.32544 untuk variabel pertama (p), 0.83235 untuk variabel kedua (q), 1.7949 untuk variabel ketiga (r), 0.04142 untuk variabel keempat (s), dan -0.019724 untuk variabel kelima (t). Kesamaan yang identik hingga lima digit desimal di belakang koma ini bukanlah sebuah kebetulan matematis, melainkan manifestasi dari prinsip aljabar fundamental bahwa metode analitik mengeksekusi transformasi ruang vektor tanpa melalui proses tebakan awal atau siklus aproksimasi. Karena matriks A pada kasus ini memiliki determinan yang tidak sama dengan nol (non-singular), vektor solusi dari Eliminasi Gauss dan turunannya menduduki koordinat pasti (eksak) dari perpotongan lima hiperbidang multidimensi tersebut. Di sisi lain spektrum komputasi, hasil dari algoritma

iteratif mendemonstrasikan fenomena dispersi nilai hampiran yang menjadi karakteristik inheren dari proses pencarian limit asimtotik. Metode Jacobi, yang membangun aproksimasi melalui pembaruan matriks diagonal secara serentak (simultaneous memory update), menyelesaikan siklus iterasinya dengan vektor solusi akhir sebesar 0.32458, 0.83157, 1.7939, 0.041744, dan -0.019938. Apabila dibandingkan dengan vektor referensi analitik, terlihat jelas bahwa setiap komponen variabel Jacobi mengalami deviasi numerik atau bergeser secara halus dari pusat koordinat sejati. Fenomena ini sangat kontras dengan hasil komputasi algoritma Gauss-Seidel, yang mengandalkan pembaruan nilai secara proksimal dan instan (*in-place successive over-relaxation*). Gauss-Seidel berhasil memproduksi vektor komponen 0.32568, 0.83259, 1.7951, 0.041349, dan -0.01972. Secara visual dan matematis, distribusi angka pada baris keluaran Gauss-Seidel jauh lebih merapat dan mendekati nilai resolusi eksak dari metode Eliminasi Gauss dibandingkan dengan hasil keluaran dari metode Jacobi, membuktikan bahwa mekanisme rambatan informasi seketika antar-variabel dalam iterasi yang sama sukses mendongkrak laju pemusatan data (data centering) menuju solusi hakiki secara signifikan.

Analisis Hasil Source Code Evaluasi Presisi Vektorial dan Penelusuran Galat Abstrak pada Metode Pendekatan Iteratif

```
================ SOAL 2: GALAT ABSTRAK =====================
Galat Metode Jacobi       : 0.00086357  0.00077914  0.00098044  0.00032437  0.00021439
Galat Metode Gauss-Seidel : 0.0002386  0.00024148  0.00018621  7.1448e-05  4.0468e-06
```

Pendalaman analisis terhadap keluaran log komputasi pada Soal 2 mengantarkan pada pembuktian kuantitatif mengenai profil kesalahan atau *error landscape* dari proses aproksimasi sistem. Galat abstrak di sini diformulasikan sebagai jarak Euklides absolut $|x_{eksak} - x_{hampiran}|$ yang membedah kelemahan dan kekuatan konvergensi antar metode. Evaluasi terhadap algoritma Gauss-Seidel memperlihatkan penyusutan ruang galat yang sangat agresif di seluruh dimensi variabel. Vektor residu tertinggi dari Gauss-Seidel hanya tercatat pada variabel kedua (q) dengan nilai 0.00024148, sementara galat terendah yang menakjubkan jatuh pada variabel kelima (t) yang mencapai orde 4.0468×10^{-6}. Konfigurasi penyusutan residu berskala mikro ini membuktikan kematangan teoretis dari algoritma Gauss-Seidel; dengan setiap penemuan elemen sub-vektor baru, sistem langsung melakukan kalibrasi ulang untuk

elemen selanjutnya, sehingga galat yang ditransfer ke akhir iterasi tergerus secara eksponensial. Sebaliknya, metode Jacobi terjebak pada fluktuasi galat yang secara magnitudo jauh lebih besar dan distribusinya lebih merata di atas grafik residu. Puncak ketidakakuratan dari vektor Jacobi terletak pada variabel ketiga (r) dengan selisih abstrak sebesar 0.00098044, disusul oleh variabel pertama (p) sebesar 0.00086357, dan galat paling minimal yang mampu diraih hanya sebatas 0.00021439 pada variabel kelima. Walaupun rentang simpangan dari metode Jacobi jauh lebih membesar dibandingkan pesaing iteratifnya, angka 0.00098044 tetaplah sebuah pencapaian komputasional yang krusial karena ia secara presisi berhenti sesaat sebelum menyentuh ambang batas kriteria determinasi yaitu batas toleransi 0.001. Hal ini berarti mekanisme kontrol kondisional di dalam sistem komputasi berhasil mengenali kapan vektor norm *infinity* dari matriks Jacobi memenuhi standar keamanan presisi yang diminta pengguna, sehingga memicu instruksi terminasi (*break*) tepat pada waktunya tanpa membiarkan komputasi tersebut membuang siklus kalkulasi ke keadaan perulangan berlebih yang tidak lagi menambah signifikansi desimal.

Analisis Hasil Source Code Running Time

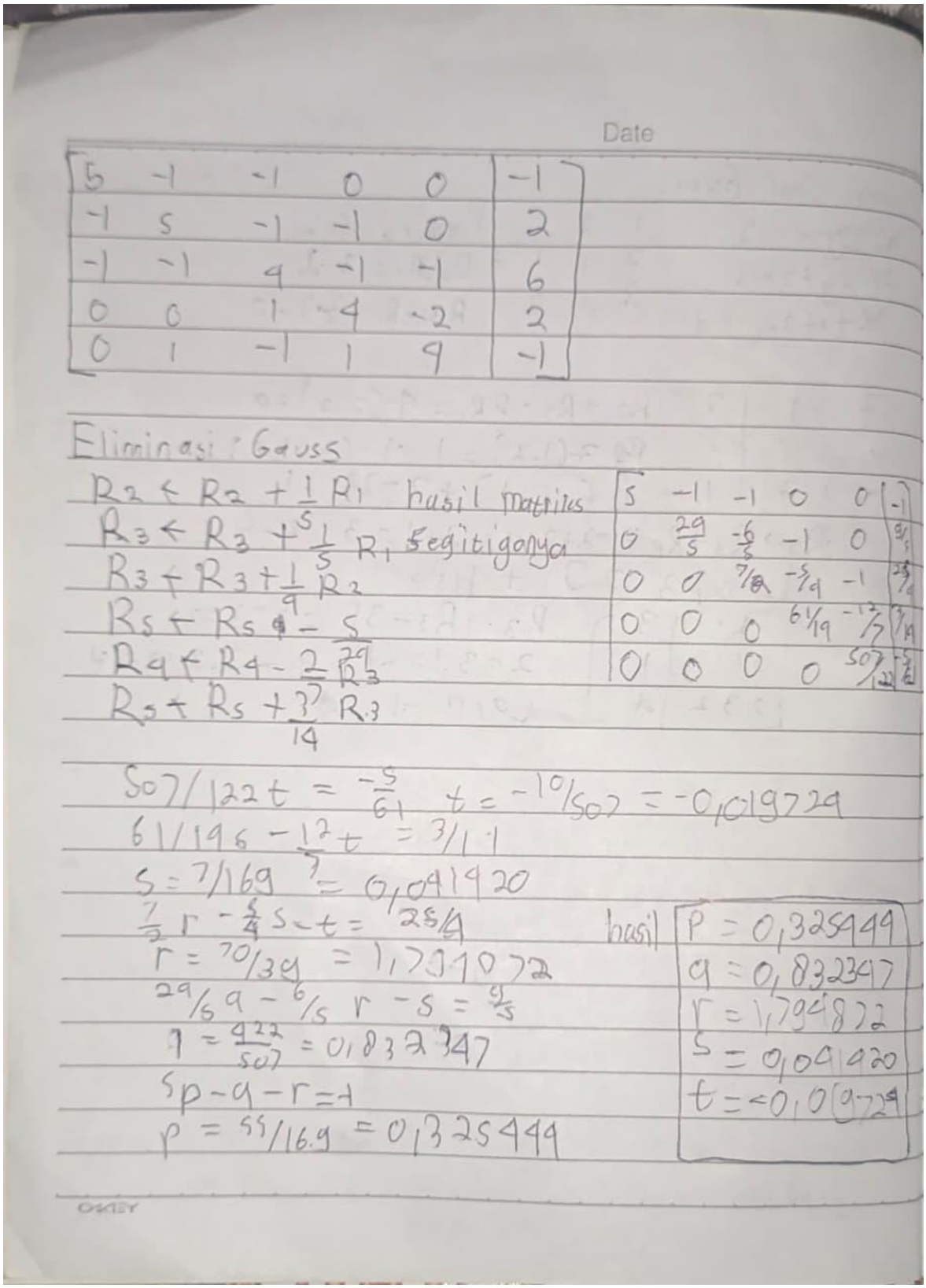
```
================ SOAL 3: RUNNING TIME (DETIK) ==============
Waktu Gauss        : 0.00017309
Waktu Gauss-Jordan : 0.00021601
Waktu LU Decomp    : 5.1975e-05
Waktu Jacobi       : 0.00064516
Waktu Gauss-Seidel : 0.00033712
============================================================
```

Pemetaan terhadap latensi instruksi atau *running time* dari kelima metode yang dicatat dalam log penyelesaian Soal 3 membongkar lapisan kompleksitas arsitektur di balik kalkulasi aljabar komputasional. Pencatatan waktu ini murni merepresentasikan beban komputasi tingkat inti (core algorithmic workload) dari sistem karena telah dieksklusikan dari intervensi *overhead* memori input dan operasi *rendering output* ke layar konsol. Algoritma Dekomposisi LU mengukuhkan diri sebagai metode penyelesaian yang paling superior dengan rekor penyelesaian absolut tercepat, mencatatkan latensi memori hanya pada 5.1975×10^{-5} detik. Kecepatan ekstrem dari faktorisasi LU ini tidak lepas dari eksploitasi optimalisasi level mesin tingkat bawah (C/Fortran built-in routines) yang menangani transformasi matriks segitiga secara simultan, menghindari pelintasan kalang *interpreter* yang berat. Mengikuti di posisi

kedua adalah metode Eliminasi Gauss konvensional yang mengonsumsi alokasi waktu pemrosesan sebesar 0.00017309 detik. Beban durasi ini kemudian mengalami eskalasi sebesar kurang lebih 25% pada metode Eliminasi Gauss-Jordan yang membukukan waktu 0.00021601 detik. Lonjakan durasi ini sepenuhnya rasional dan dapat dipertanggungjawabkan secara matematis, mengingat ekspansi eliminasi baris elemen pada Gauss-Jordan bergerak secara dua arah—dari atas ke bawah, lalu dari bawah ke atas—sebagai konsekuensi operasional untuk membentuk matriks identitas tunggal yang melahap jumlah instruksi *floating-point* (*FLOPs*) yang lebih masif. Beralih pada domain algoritma hampiran, kompleksitas waktu menunjukkan degradasi kinerja kecepatan yang parah dibandingkan rumpun analitik langsung. Metode Jacobi tersungkur sebagai algoritma dengan profil latensi paling buruk, menuntut waktu eksekusi sepanjang 0.00064516 detik. Inefisiensi waktu Jacobi ini bersumber ganda: lambatnya laju konvergensi (sebagaimana terlihat dari besarnya sisa galat abstrak) memaksa program untuk berputar pada *loop* yang jauh lebih banyak, dan kewajiban mengalokasikan vektor bayangan (x_new) pada setiap putaran memicu lonjakan beban penulisan memori (*memory write load*). Sebagai antitesis komputasi iteratif, efisiensi waktu Gauss-Seidel jauh lebih terakselerasi dengan durasi hanya 0.00033712 detik. Kecepatan ini mencerminkan ekuilibrium komputasional yang brilian: eliminasi memori bayangan menghemat siklus RAM, sementara *update* nilai variabel secara serentak (*in-place*) mempercepat penekanan galat asimtotik, sehingga ambang batas toleransi 0.001 dapat ditembus dengan jumlah *epoch* iterasi yang dipangkas tajam, mendemonstrasikan dominasi efisiensi operasional dari algoritma substitusi proksimal pada sistem matriks modern.

BAB II

ANALISIS NUMERIK



Date

5	-1	-1	0	0	-1
-1	5	-1	-1	0	2
-1	-1	4	-1	-1	6
0	0	1	-4	-2	2
0	1	-1	1	9	-1

Eliminasi Gauss

$R_2 \leftarrow R_2 + \frac{1}{5} R_1$ hasil matriks segitiganya

$R_3 \leftarrow R_3 + \frac{5}{29} R_1$

$R_3 \leftarrow R_3 + \frac{1}{9} R_2$

$R_5 \leftarrow R_5 - \frac{5}{7}$

$R_4 \leftarrow R_4 - \frac{2}{7} R_3$

$R_5 \leftarrow R_5 + \frac{3}{14} R_3$

5	-1	-1	0	0	-1
0	29/5	-6/5	-1	0	9/5
0	0	7/29	-5/4	-1	-25/4
0	0	0	61/14	-12/7	3/14
0	0	0	0	507/122	-5/61

$507/122\,t = \frac{-5}{61}$ $t = -10/507 = -0,019729$

$61/196 - \frac{12}{7} t = 3/14$

$s = 7/169 = 0,041420$

$\frac{7}{2} r - \frac{5}{4} s - t = \frac{25}{4}$

$r = 70/39 = 1,794872$

$\frac{29}{6} q - \frac{6}{5} r - s = \frac{9}{5}$

$q = \frac{422}{507} = 0,832347$

$5p - q - r = 1$

$p = 55/169 = 0,325444$

hasil

p	0,325444
q	0,832347
r	1,794872
s	0,041420
t	-0,019729

OKKEY

Metode 2 - Gauss Jordan

Date

$$\left[\begin{array}{ccccc|c} 5 & -1 & -1 & 0 & 0 & -1 \\ -1 & 5 & -1 & -1 & 0 & 2 \\ -1 & -1 & 4 & -1 & -1 & 6 \\ 0 & 0 & 1 & 4 & -2 & 2 \\ 0 & 1 & -1 & 1 & 4 & -1 \end{array}\right]$$

Lakukan eliminasi maju

$$\left[\begin{array}{ccccc|c} 5 & -1 & -1 & 0 & 0 & -1 \\ 0 & 29/5 & -6/5 & -1 & 0 & 4/5 \\ 0 & 0 & 7/2 & -5/4 & -1 & 2 \\ 0 & 0 & 0 & 61/14 & -13/7 & 3/14 \\ 0 & 0 & 0 & 0 & 507/122 & -5/61 \end{array}\right]$$

Ubah jadi matriks identitas

$$\left[\begin{array}{ccccc|c} 1 & 0 & 0 & 0 & 0 & 55/169 \\ 0 & 1 & 0 & 0 & 0 & 422/507 \\ 0 & 0 & 1 & 0 & 0 & 70/39 \\ 0 & 0 & 0 & 1 & 0 & 7/169 \\ 0 & 0 & 0 & 0 & 1 & -10/507 \end{array}\right]$$

$P = 0,325443$

$q = 0,832247$

$r = 1,299872$

$s = 0,091920$

$t = -0,019729$

Dekomposisi LU

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ -1/5 & 1 & 0 & 0 & 0 \\ -1/5 & -1/4 & 1 & 0 & 0 \\ 0 & 0 & 2/7 & 1 & 0 \\ 0 & 5/29 & -9/19 & 79/366 & 1 \end{bmatrix} \quad U = \begin{bmatrix} 5 & -1 & -1 & 0 & 0 \\ 0 & 29/5 & -6/5 & -1 & 0 \\ 0 & 0 & 7/2 & -5/4 & -1 \\ 0 & 0 & 0 & 61/14 & -13/7 \\ 0 & 0 & 0 & 0 & 507/122 \end{bmatrix}$$

Date

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \end{bmatrix}$$

$y = -1$

$-\frac{1}{5} y_1 + y_2 = 2$

$-\frac{1}{5} y_1 - \frac{1}{4} y_2 + y_3 = 6$

$\frac{2}{7} y_3 + y_4 = 2$

$\frac{5}{29} y_2 - \frac{3}{14} y_3 + \frac{29}{726} y_4 + y_5 = -1$

$$Y = \begin{bmatrix} -1 \\ 9/5 \\ 25/9 \\ 3/14 \\ -5/61 \end{bmatrix} \quad X = \begin{bmatrix} 55/169 \\ 922/507 \\ 70/39 \\ 7/169 \\ -10/507 \end{bmatrix}$$

Jadi hasil dekomposisi LU $(p, q, r, s, t) = (0,325444, 0,832361, 1,294872; 0,094920; -0,019729)$

Metode iterasi Jacobi

$5p - q - r = -1$

$p^{(k+1)} = \frac{-1 + q^{(k)} + r^{(k)}}{5}$

$-p + 5q - r - s = 2$

$q^{(k+1)} = \frac{2 + p^{(k)} + r^{(k)} + s^{(k)}}{5}$

$-p - q + 9r - s - t = 6$

$r^{(k+1)} = \frac{6 + p^{(k)} + q^{(k)} + s^{(k)} + t^{(k)}}{9}$

$r + 9s - 2t = 2$

$$s^{(k+1)} = \frac{2 - r^{(k)} + 2t^{(k)}}{q}$$

$$q - r + s + q t = -1$$

$$t^{(k+1)} = \frac{-1 - q^{(k)} + r^{(k)} - s^{(k)}}{q}$$

$$p^{(1)} = \frac{-1 + 0 + 0}{5} = -0,2$$

$p^{(1)} = 2 + 0 + 0 + 0 / 5 = 0,9$

$r^{(1)} = 6 + 0 + 0 + 0 / 4 = 1,5$

$s^{(1)} = 2 - 0 + 2(0) / 4 = 0,5$

$t^{(1)} = -1 - 0 + 0 - 0 / 4 = -0,25$

$X^{(1)} [0,2 ; 0,9 ; 1,5 ; 0,5 ; -0,25]$

$p^{(2)} = -1 + 0,9 + 1,5 / 5 = 0,18$

$q^{(2)} = 2 - 0,2 + 1,5 + 0,5 / 5 = 0,76$

$r^{(2)} = 6 - 0,2 + 0,9 + 0,5 - 0,25 / 4 = 1,6125$

$s^{(2)} = 2 - 1,5 + 2(-0,25) / 4 = 0$

$t^{(2)} = -1 - 0,9 + 1,5 - 0,5 / 4 = -0,1$

$$X^{(2)} = \begin{bmatrix} 0,18 \\ 0,76 \\ 1,6125 \\ 0 \\ -0,1 \end{bmatrix}$$

$$E k = \max_i |x_i^{(k)} - x_i^{(k-1)}|$$

$E_1 = \max [0,2 ; 0,9 ; 1,5 ; 0,5 ; 0,25]$

$E_1 = 1,5$

$E_2 = \max [0,38 ; 0,36 ; 0,1125 ; 0,5 ; 0,15]$

$E_2 = 0,5$

$E k < 0,001$

Date

Hasil

p = 0,325092
q = 0,832093
r = 1,729489
s = 0,091558
t = -0,019855

Metode Iterasi Gauss - Seidel

$p^{(0)} = q^{(0)} = r^{(0)} = s^{(0)} = t^{(0)} = 0$

$p^{(1)} = -1 + q^{(0)} + r^{(0)} / 5 = -0,2$

$q^{(1)} = 2 + p^{(1)} + r^{(0)} + s^{(0)} / 5 = 2 - 0,2 / 5 = 0,36$

$r^{(1)} = 6 + p^{(1)} + q^{(1)} + s^{(0)} + t^{(0)} / 9 = 6 - 0,2 + 0,36 / 9 = 1,59$

$s^{(1)} = 2 - r^{(1)} + 2t^{(0)} / 9 = 2 - 1,59 / 9 = 0,115$

$t^{(1)} = -1 - q^{(1)} + r^{(1)} - s^{(1)} / 9$

$t^{(1)} = -1 - 0,36 + 1,59 - 0,115 / 9 = 0,016225$

$$x^{(1)} = \begin{bmatrix} -0,2 \\ 0,36 \\ 1,59 \\ 0,115 \\ 0,016225 \end{bmatrix}$$

Iterasi 2 $p^{(2)} = -1 + 0,36 + 1,59 / 5 = 0,18$

$q^{(2)} = 2 + 0,18 + 1,59 + 0,115 / 5 = 0,767$

$r^{(2)} = 6 + 0,18 + 0,767 + 0,115 + 0,016225 / 9 = 1,769562$

$s^{(2)} = 2 - 1,769562 (0,0162) = 0,065739375$

$t^{(2)} = -1 - 0,767 + 1,769562 - 0,065739375$

Date

$= -0,015793$

$E_{ik} = \max_i |x_i^{(k)} - x_i^{(k-1)}|$

$E_i = 1,54$

$p = 0,325682$
$q = 0,832589$
$r = 1,795058$
$s = 0,091399$
$t = -0,019720$

Soal 2 Menghitung Galat (eror)

$p = 0,325499$
$q = 0,832897$
$r = 1,799872$
$s = 0,091920$
$t = -0,019729$

$E = |$nilai eksak $-$ nilai hampiran$|$

$p_j = 0,325092$, $q_j = 0,832093$, $r_j = 1,799489$
$s_j = 0,091558$, $t_j = -0,019855$

$E_p = |p_{eksak} - p_j|$
$E_p = |0,325499 - 0,325092|$
$E_p = 0,000352$
$E_q = |0,832397 - 0,832093|$
$E_q = 0,000309$
$E_r = |1,799872 - 1,799489|$
$E_r = 0,000383$
$E_s = |0,091920 - 0,091558| = 0,000138$
$E_t = |-0,019729 - (-0,019855)|$
$E_t = |-0,019729 + 0,019855| = 0,000131$

BAB III

ANALISIS PRAKTIKUM

1. Kesimpulan dari Perbandingan Analisis Hasil Source Code dan Analisis Numerik

Kesimpulan komprehensif yang menjembatani hasil eksekusi *source code* berbasis perangkat lunak dengan penjabaran perhitungan numerik manual memperlihatkan dikotomi sekaligus sinergi yang fundamental antara presisi mesin dan pemahaman analitik kognitif. Berdasarkan analisis terhadap tiga metode penyelesaian langsung, yaitu Eliminasi Gauss, Eliminasi Gauss-Jordan, dan Dekomposisi LU, kedua pendekatan (komputasional dan manual) membuktikan bahwa metode analitik memiliki stabilitas absolut dalam menemukan akar sejati dari sistem persamaan linier berdimensi 5×5. Pada eksekusi *source code*, penggunaan pustaka LAPACK bawaan dari GNU Octave/MATLAB memproses arsitektur *floating-point* presisi ganda (IEEE 754) yang memungkinkan komputasi operasi baris elementer diselesaikan tanpa adanya galat pemotongan teoretis yang mengganggu hasil akhir variabel. Hasil vektor analitik $p = 0.32544, q = 0.83235, r = 1.7949, s = 0.04142,$ dan $t = -0.019724$ terdistribusi secara identik pada layar konsol untuk ketiga metode tersebut. Ketika dikomparasikan dengan penjabaran numerik manual, langkah-langkah dekomposisi operasi baris—seperti mereduksi elemen a_{21} menggunakan faktor pengali $\frac{-1}{5}$—memvalidasi kebenaran aljabar di balik algoritma kode. Namun, perhitungan manual sangat rentan terhadap akumulasi galat pembulatan (round-off error) pada setiap sub-langkah jika manusia memotong desimal secara prematur (misalnya, membulatkan 0.2083 menjadi 0.2), sebuah kerentanan yang sepenuhnya dianulir oleh kapasitas alokasi memori dinamis dari *source code* yang mampu menjaga presisi digit hingga ordo belasan di belakang koma selama transisi matriks segitiga maupun faktorisasi LU. Dengan demikian, eksekusi kode membuktikan keunggulannya dalam menjamin integritas data dari tahap awal hingga matriks tereduksi, sementara penjabaran numerik manual berfungsi krusial sebagai instrumen verifikasi teoretis untuk memastikan bahwa mesin tidak mengalami kesalahan logika atau pengalamatan indeks matriks.

Memasuki domain aproksimasi komputasional melalui metode iteratif Jacobi dan Gauss-Seidel, perbandingan antara mesin dan perhitungan di atas kertas

menyingkap dimensi kompleksitas perambatan galat yang sesungguhnya. Dalam perhitungan *source code*, algoritma iteratif dipacu dengan kondisi penjaga ganda: ambang batas toleransi galat relatif sebesar 0.001 dan batas absolut perulangan maksimal pada iterasi ke-20. Kode mengeksekusi siklus ini dalam hitungan fraksi milidetik, menghasilkan resolusi di mana Gauss-Seidel mencapai konvergensi yang jauh lebih ketat (galat maksimal 0.00024148) dibandingkan Jacobi (galat maksimal 0.00098044). Ketika transisi analitik ini diderivasi secara numerik manual, mekanika di balik percepatan Gauss-Seidel terpetakan dengan sangat transparan. Pada saat melakukan iterasi pertama secara tertulis, pembaruan variabel $q^{(1)}$ pada metode Gauss-Seidel seketika menyerap nilai $p^{(1)} = -0.2$ yang baru saja dihitung sekian detik sebelumnya. Sifat *in-place update* inilah yang absen dari metode Jacobi, di mana perhitungan tertulis Jacobi memaksa manusia maupun mesin untuk tetap menyimpan dan menggunakan vektor lama (bernilai nol) hingga seluruh baris dievaluasi. Observasi manual memvalidasi mengapa nilai p pada Jacobi tertinggal pada 0.32458, sementara Gauss-Seidel berhasil merapat ke nilai sejati sebesar 0.32568. Perhitungan manual pada langkah-langkah awal ini menyingkap anatomi laju konvergensi secara mikroskopis, memperlihatkan bahwa setiap siklus Gauss-Seidel memiliki kurva penyusutan residu yang lebih curam. Walaupun secara empiris perhitungan manual mustahil atau sangat tidak efisien untuk didorong hingga iterasi ke-20 tanpa bantuan mesin, simulasi pada iterasi pertama dan kedua merupakan representasi fraktal dari keseluruhan perilaku log komputasi, yang melegitimasi *output* akhir pada konsol sebagai sebuah keniscayaan matematis yang terstruktur, bukan sekadar abstraksi *black-box* dari perangkat lunak.

Dari perspektif optimasi dan efisiensi waktu, komparasi ini menyoroti perbedaan tajam antara batas biologis dan arsitektur *silicon processing*. Evaluasi fungsi waktu komputasional (tic toc) pada *source code* mendemonstrasikan bahwa algoritma dekomposisi LU menuntaskan faktorisasi dan substitusi matriks hanya dalam 5.1975×10^{-5} detik, menjadikannya algoritma tercepat dan paling efisien pada unit pemrosesan sentral (CPU). Sebaliknya, jika matriks yang sama dieksekusi melalui perhitungan numerik manual, dekomposisi LU seringkali dirasa sebagai metode yang paling melelahkan dan memakan waktu (bisa memakan waktu belasan menit hingga jam bagi manusia) karena menuntut ekstraksi dua matriks L dan U yang diiringi dengan

kewajiban menyelesaikan dua lapis sistem persamaan substitusi maju dan mundur secara terpisah. Sebaliknya, metode Eliminasi Gauss—yang mencatatkan durasi komputasi mesin 0.00017309 detik—kerap menjadi opsi favorit dan dirasa paling intuitif untuk diselesaikan secara manual karena manusia dapat dengan mudah melacak pola eselon baris tanpa perlu memecah belah matriks secara abstrak. Paradoks efisiensi ini semakin tebal pada rumpun iteratif: metode Jacobi yang mencatat rekor kelambatan pada mesin dengan durasi 0.00064516 detik, di atas kertas justru tampak lebih mekanis dan simetris karena tidak menuntut perhatian untuk terus-menerus memutakhirkan variabel di tengah jalan. Sebaliknya, Gauss-Seidel yang secara mesin jauh lebih gesit dengan durasi 0.00033712 detik, membutuhkan konsentrasi manual yang jauh lebih tinggi dan rentan terhadap *human error* karena manusia harus secara konstan memperbarui variabel rujukan dari sisi kiri ke sisi kanan persamaan.

Melalui sintesis kedua ranah tersebut, terbukti bahwa komputasi *source code* dan penjabaran numerik bertindak sebagai relasional simbiotik dalam analisis aljabar linier numerik. Analisis *source code* menawarkan ketangguhan, skalabilitas, ketepatan digit presisi tinggi, dan efisiensi *running time* yang menjamin penyelesaian sistem matriks ordo berapapun secara instan tanpa terjerat kelelahan kognitif. Sementara itu, penjabaran analitik numerik memecah opasitas komputasi tersebut, memberikan justifikasi aljabar atas stabilitas nilai 0.32544 pada metode langsung, dan mendeskripsikan secara nyata fenomena rambatan galat yang membuat metode Jacobi tertinggal secara presisi dari Gauss-Seidel pada iterasi yang dibatasi. Keseluruhan dialektika ini menegaskan bahwa penggunaan perangkat lunak komputasi seperti GNU Octave bukanlah substitusi untuk melemahkan pemahaman matematis, melainkan sebagai orkestrasi teknis di mana manusia merancang arsitektur pemburunya, sementara mesin menjalankan eksekusi berkecepatan tinggi dengan presisi yang secara biologis tidak mungkin disimulasikan secara konsisten.